

# Skin Cancer Classifier

# Table of Contents

|    |                                       |
|----|---------------------------------------|
| 01 | Introduction                          |
| 02 | Dataset and Exploratory Data Analysis |
| 03 | Data Preprocessing                    |
| 04 | Model                                 |
| 05 | Frontend Demonstration                |
| 06 | Future Improvements                   |
| 07 | Conclusion                            |

01

# Introduction

# Project Goal

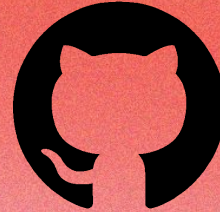
Our goal is to make a model that  
can classify images of different  
skin cancer lesions

# Importance

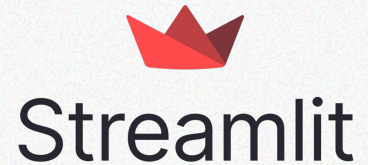
- Most common form of cancer
- Improves survival rate by early detections
- Reduce time consuming diagnostics
- Improves accuracy

# Tools Used

- Github
- Google Colab
- Kaggle
- Streamlit
- Numpy
- Matplotlib



kaggle



02

Dataset

# Dataset

Kaggle: Skin Cancer MNIST: HAM10000

## Dataset Size:

---

- Entire dataset size: 2.9GB
- HAM10000: "Human Against Machine with 10000 training images"
- 10015 dermatoscopic images of pigmented lesions:
  - ex: basal cell carcinoma (bcc), melanoma (mel), melanocytic nevi (nv)
- 50% of lesions were confirmed through histopathology

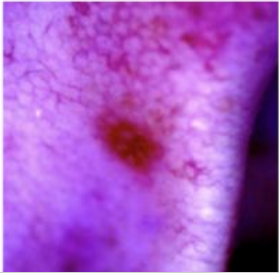
## Column Names:

---

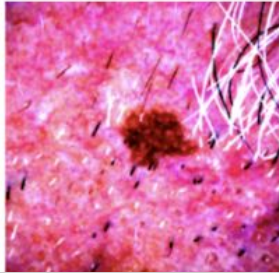
- lesion\_id
- image\_id
- dx
- dx\_type
- age
- sex
- localization

# Pigmented Lesions

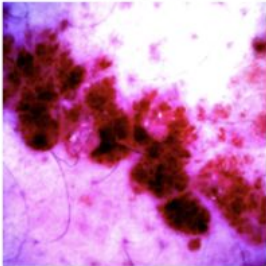
Label: nv



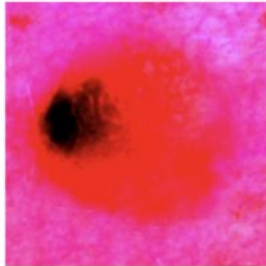
Label: bkl



Label: bkl

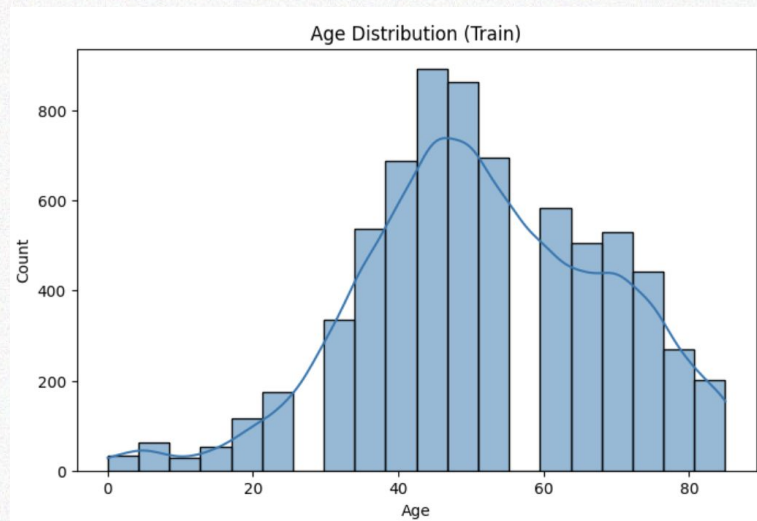
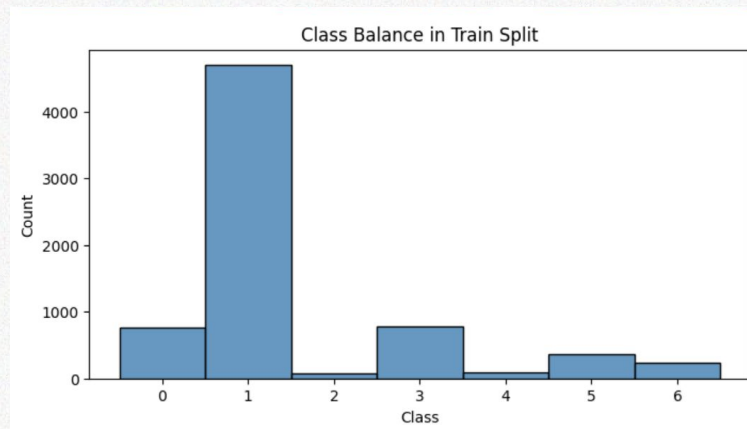
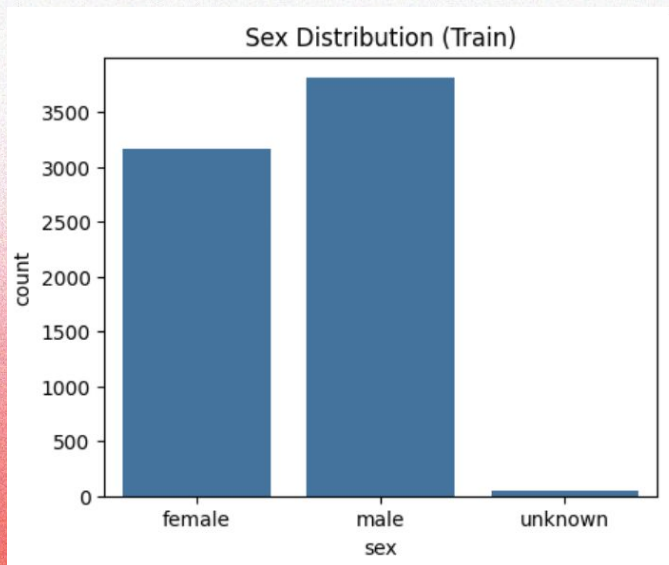


Label: nv



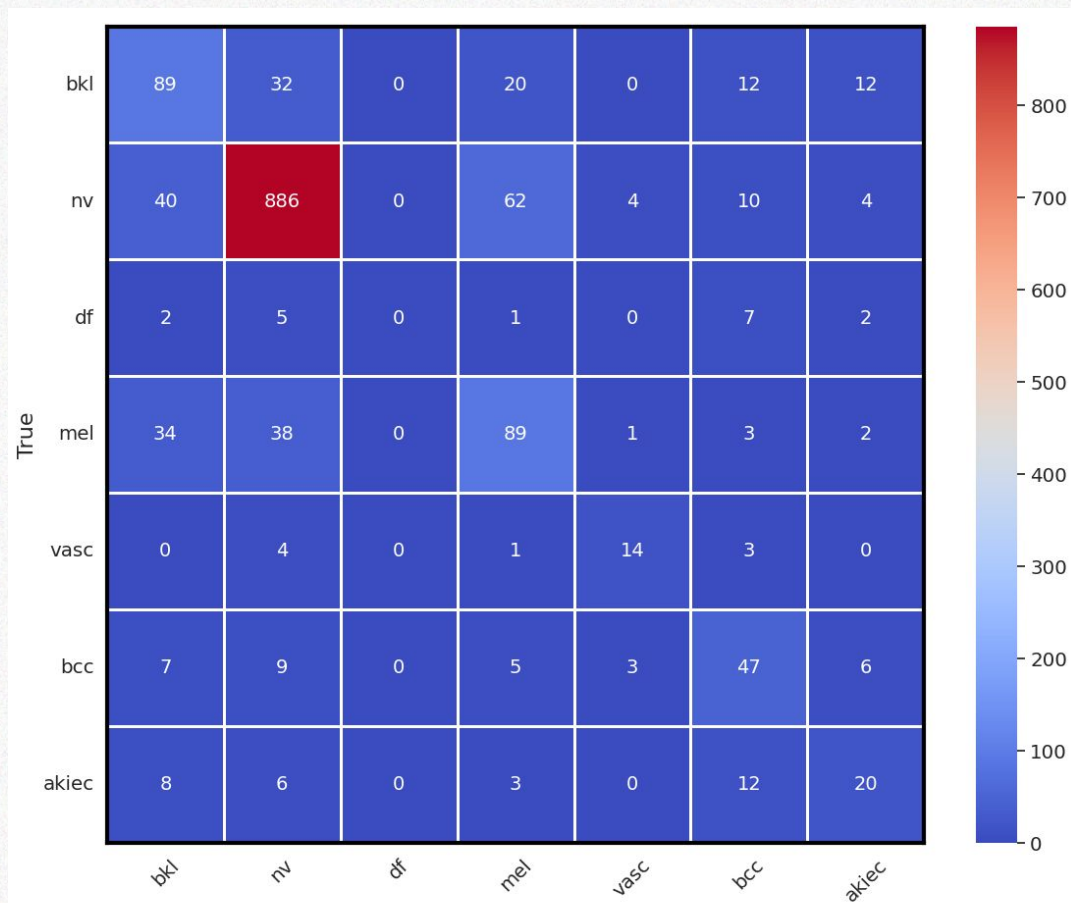
- Actinic keratoses and intraepithelial carcinoma / Bowen's disease (akiec)
- Basal cell carcinoma (bcc)
- Benign keratosis-like lesions (solar lentigines / seborrheic keratoses and lichen-planus like keratoses, bkl)
- Dermatofibroma (df)
- Melanoma (mel)
- Melanocytic nevi (nv)
- Vascular lesions (angiomas, angiokeratomas, pyogenic granulomas and hemorrhage, vasc)

# Exploratory Data Analysis



# Correlation Matrix

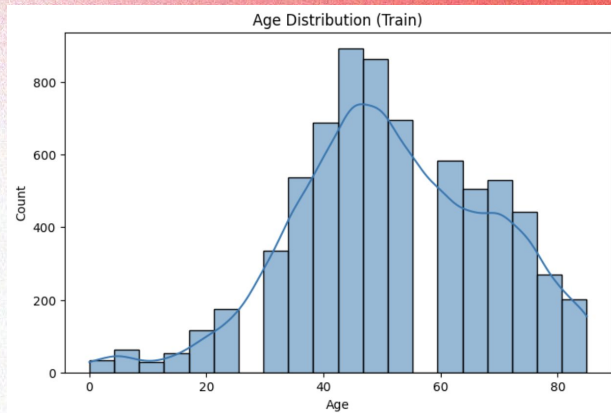
- Highest class prediction for nevus
- Smaller classes have reduced recall
- Melanoma is often misidentified as nevus
  - Indicates bias towards dominant class
- Overall accuracy is driven by overrepresentation of the majority class nv



03

# Data Preprocessing

# Data Preprocessing



```
train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

## Cleaning and Feature Engineering

- Filled missing ages with median
- Created age\_missing indicator feature
- Stratified dataset split:
  - 70% Training
  - 15% Validation
  - 15% Test

## Image Transformations

- Improve generalization and model stability

04

Model

# CNN Architecture

## Dual-Branch Network

- Model built from scratch
  - 3 convolutional layers
  - ReLU + Max Pooling
  - Fully connected layer
  - Dropout (0.5)

```
class SkinCancerCNN(nn.Module):
    def __init__(self, num_classes):
        super(SkinCancerCNN, self).__init__()

        # ----- CNN for image -----
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 112x112

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 56x56

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 28x28
        )

        self.image_fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 28 * 28, 512),
            nn.ReLU(),
            nn.Dropout(0.5)
        )

        # ----- Metadata branch -----
        self.meta_fc = nn.Sequential(
            nn.Linear(2, 32),
            nn.ReLU()
        )

        # ----- Final classifier -----
        self.classifier = nn.Linear(512 + 32, num_classes)

    def forward(self, image, metadata):
        img_features = self.features(image)
        img_features = self.image_fc(img_features)

        meta_features = self.meta_fc(metadata)

        combined = torch.cat((img_features, meta_features), dim=1)
        output = self.classifier(combined)
```

# Training Results

- **Total Epochs: 10**

Overall:

- Training Accuracy: 79%
- Validation Accuracy: 77%

- **Training**

- Performance Trend
- Training loss decreased steadily
- Validation loss decreased consistently
- Accuracy improved across epochs

```
... Epoch 1/10
Train Loss: 1.1856, Train Acc: 0.6335
Val Loss: 0.8588, Val Acc: 0.6884
-----
Epoch 2/10
Train Loss: 0.8119, Train Acc: 0.7141
Val Loss: 0.7426, Val Acc: 0.7290
-----
Epoch 3/10
Train Loss: 0.7329, Train Acc: 0.7305
Val Loss: 0.6854, Val Acc: 0.7437
-----
Epoch 4/10
Train Loss: 0.6959, Train Acc: 0.7448
Val Loss: 0.6910, Val Acc: 0.7497
-----
Epoch 5/10
Train Loss: 0.6678, Train Acc: 0.7625
Val Loss: 0.6554, Val Acc: 0.7617
-----
Epoch 6/10
Train Loss: 0.6476, Train Acc: 0.7631
Val Loss: 0.6627, Val Acc: 0.7610
-----
Epoch 7/10
Train Loss: 0.6246, Train Acc: 0.7706
Val Loss: 0.6226, Val Acc: 0.7670
-----
Epoch 8/10
Train Loss: 0.6014, Train Acc: 0.7819
Val Loss: 0.6408, Val Acc: 0.7617
-----
Epoch 9/10
Train Loss: 0.5776, Train Acc: 0.7929
Val Loss: 0.6090, Val Acc: 0.7716
-----
Epoch 10/10
Train Loss: 0.5734, Train Acc: 0.7934
Val Loss: 0.5902, Val Acc: 0.7763
-----
```

05

# Frontend Demonstration

# Streamlit App

06

# Future Improvements

# Future Improvements

- The dataset is highly imbalanced
  - Other skin lesion classes require more samples
  - Apply class weighting
  - Can help improve efficiency in classifying different types of skin cancer
- Increase dataset diversity across skin tones
- Real World Applications:
  - Early melanoma screening in primary care settings
  - Educational tool for medical training

Thank You!